

✓ Suicide Detection - BERT Fine-Tuning

This notebook fine-tunes BERT (bert-base-uncased) on the suicide detection dataset.

```
!pip install -q kaggle
from google.colab import files
files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d nikhileswarkomati/suicide-watch
!unzip suicide-watch.zip
import pandas as pd
df = pd.read_csv('Suicide_Detection.csv')
print(f"Dataset shape: {df.shape}")
print(df.head())
```



Choose Files

 No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle (1).json
Dataset URL: <https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch>
License(s): CC-BY-SA-4.0
suicide-watch.zip: Skipping, found more recently modified local copy (use --force to force download)
Archive: suicide-watch.zip
replace Suicide_Detection.csv? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
inflating: Suicide_Detection.csv
Dataset shape: (232074, 3)

Unnamed: 0		text	class
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3	Am I weird I don't get affected by compliments...	non-suicide
2	4	Finally 2020 is almost over... So I can never ...	non-suicide
3	8	i need helpjust help me im crying so hard	suicide
4	9	I'm so lostHello, my name is Adam (16) and I've...	suicide

```
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np
import time
import os
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

print("Loading dataset...")
df = pd.read_csv("Suicide_Detection.csv")
df = df.dropna()
df['label'] = df['class'].map({'non-suicide': 0, 'suicide': 1})
print(f"Dataset shape: {df.shape}")
print(f"Class distribution: {df['label'].value_counts()}")

train_texts, val_texts, train_labels, val_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(), test_size=0.2, stratify=df['label'], random_state=42
)

print("Tokenizing data...")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

print("Tokenizing training data...")
train_encodings = tokenizer(
    [text for text in tqdm(train_texts, desc="Tokenizing training texts")],
    truncation=True,
    padding='max_length',
    max_length=256
)

print("Tokenizing validation data...")
val_encodings = tokenizer(
    [text for text in tqdm(val_texts, desc="Tokenizing validation texts")],
    truncation=True,
    padding='max_length',
    max_length=256
)

class SuicideDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = SuicideDataset(train_encodings, train_labels)
val_dataset = SuicideDataset(val_encodings, val_labels)

batch_size = 16
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)

print("Loading BERT model...")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
model.to(device)

optimizer = AdamW(model.parameters(), lr=2e-5, eps=1e-8)

epochs = 3
total_steps = len(train_loader) * epochs
```

```
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)

print(f"Starting training for {epochs} epochs...")
best_val_loss = float('inf')

for epoch in range(epochs):
    print(f"\nEpoch {epoch+1}/{epochs}")

    model.train()
    total_train_loss = 0

    train_progress_bar = tqdm(train_loader, desc="Training", leave=True)

    for batch in train_progress_bar:
        optimizer.zero_grad()

        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss

        total_train_loss += loss.item()
        train_progress_bar.set_postfix({'loss': f"{loss.item():.4f}"})

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()

    avg_train_loss = total_train_loss / len(train_loader)
    print(f"Average training loss: {avg_train_loss:.4f}")

    model.eval()
    total_val_loss = 0
    preds, true = [], []

    val_progress_bar = tqdm(val_loader, desc="Validation", leave=True)

    for batch in val_progress_bar:
        batch = {k: v.to(device) for k, v in batch.items()}

        with torch.no_grad():
            outputs = model(**batch)

        loss = outputs.loss
        total_val_loss += loss.item()

        logits = outputs.logits
        preds.extend(torch.argmax(logits, dim=1).cpu().numpy())
        true.extend(batch['labels'].cpu().numpy())

    avg_val_loss = total_val_loss / len(val_loader)
    print(f"Validation loss: {avg_val_loss:.4f}")

    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        print("Saving best model...")
        model_path = "best_suicide_detection_model"
        model.save_pretrained(model_path)
        tokenizer.save_pretrained(model_path)

    print("\nClassification Report:")
    report = classification_report(true, preds, target_names=['Non-suicide', 'Suicide'])
    print(report)

print("Training complete!")

best_model = BertForSequenceClassification.from_pretrained("best_suicide_detection_model")
best_model.to(device)

def predict_suicide_risk(texts, model=best_model, tokenizer=tokenizer):
    model.eval()
    encodings = tokenizer(texts, truncation=True, padding=True, max_length=256, return_tensors="pt")
    encodings = {k: v.to(device) for k, v in encodings.items()}

    with torch.no_grad():
        outputs = model(**encodings)

    probs = torch.nn.functional.softmax(outputs.logits, dim=1)
    predictions = torch.argmax(probs, dim=1)

    results = []
    for i, text in enumerate(texts):
        results.append({
            'text': text,
            'prediction': 'suicide' if predictions[i].item() == 1 else 'non-suicide',
            'confidence': probs[i][predictions[i]].item()
        })

    return results

example_texts = [
    "I feel like everything is fine today",
    "I don't see any point in continuing anymore"
]

if os.path.exists("best_suicide_detection_model"):
    results = predict_suicide_risk(example_texts)
    for result in results:
        print(f"Text: {result['text']}")
        print(f"Prediction: {result['prediction']}")
        print(f"Confidence: {result['confidence']:.4f}")
        print()
```

```
↳ Using device: cuda
Loading dataset...
Dataset shape: (232074, 4)
Class distribution: label
1    116037
0    116037
Name: count, dtype: int64
Tokenizing data...
tokenizer_config.json: 100%          48.0/48.0 [00:00<00:00, 5.15kB/s]
vocab.txt: 100%                    232k/232k [00:00<00:00, 15.9MB/s]
tokenizer.json: 100%                466k/466k [00:00<00:00, 41.4MB/s]
config.json: 100%                   570/570 [00:00<00:00, 52.6kB/s]
Tokenizing training data...
Tokenizing training texts: 100%      185659/185659 [00:00<00:00, 2414650.46it/s]
Tokenizing validation data...
Tokenizing validation texts: 100%    46415/46415 [00:00<00:00, 1107506.61it/s]
Loading BERT model...
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the pa
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
model.safetensors: 100%             440M/440M [00:01<00:00, 491MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Starting training for 3 epochs...

Epoch 1/3
Training: 100%                      11604/11604 [29:51<00:00, 6.65it/s, loss=0.0008]
Average training loss: 0.1031
Validation: 100%                    2901/2901 [02:28<00:00, 19.49it/s]
Validation loss: 0.0781
Saving best model...

Classification Report:
      precision    recall  f1-score   support

Non-suicide      0.98      0.97      0.98     23208
Suicide          0.97      0.98      0.98     23207

   accuracy
macro avg      0.98      0.98      0.98     46415
weighted avg   0.98      0.98      0.98     46415

Epoch 2/3
Training: 100%                      11604/11604 [29:51<00:00, 6.67it/s, loss=0.0002]
Average training loss: 0.0503
Validation: 100%                    2901/2901 [02:28<00:00, 19.60it/s]
Validation loss: 0.0820

Classification Report:
      precision    recall  f1-score   support

Non-suicide      0.98      0.98      0.98     23208
Suicide          0.98      0.98      0.98     23207

   accuracy
macro avg      0.98      0.98      0.98     46415
weighted avg   0.98      0.98      0.98     46415

Epoch 3/3
Training: 100%                      11604/11604 [29:50<00:00, 6.70it/s, loss=0.0005]
Average training loss: 0.0191
Validation: 100%                    2901/2901 [02:27<00:00, 19.63it/s]
Validation loss: 0.1095

Classification Report:
      precision    recall  f1-score   support

Non-suicide      0.98      0.98      0.98     23208
Suicide          0.98      0.98      0.98     23207

   accuracy
macro avg      0.98      0.98      0.98     46415
weighted avg   0.98      0.98      0.98     46415

Training complete!
Text: I feel like everything is fine today
Prediction: non-suicide
Confidence: 0.9949

Text: I don't see any point in continuing anymore
Prediction: suicide
Confidence: 0.9969
```

```
import pandas as pd

final_metrics = {
    "Non-suicide": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 23208},
    "Suicide": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 23207},
    "accuracy": 0.98,
    "macro avg": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 46415},
}
pd.DataFrame(final_metrics).T[["precision", "recall", "f1-score", "support"]]
```

	precision	recall	f1-score	support
Non-suicide	0.98	0.98	0.98	23208.00
Suicide	0.98	0.98	0.98	23207.00
accuracy	0.98	0.98	0.98	0.98